

Practice Exam

Databricks Certified Data Engineer Associate - Advanced Level

Overview

This is an advanced practice exam for the Databricks Certified Data Engineer Associate exam. The questions here are designed to be more challenging than the actual exam to ensure comprehensive preparation. After taking this practice exam, you should be well-prepared for the actual Data Engineer Associate exam.

Just like the actual exam, it contains 100 multiple-choice questions. Each of these questions has one correct answer. The correct answer for each question is listed at the bottom in the **Correct Answers** section.

Important Information:

- **Time Limit:** 120 minutes (recommended for advanced practice)
- **Passing Score:** 70 out of 100 questions (70%)
- **Format:** Multiple choice with 5 options (A, B, C, D, E)
- **No External Resources:** No documentation or Databricks environments allowed during exam
- **Question Distribution:** Questions cover all exam domains with increased complexity

Exam Domains Covered:

1. Databricks Lakehouse Platform (15%)
2. ELT with Spark SQL and Python (30%)
3. Incremental Data Processing (20%)

4. Production Pipelines (20%)

5. Data Governance (15%)

If you have questions about this practice exam, please review the official Databricks Academy Certification materials and documentation.

Questions

Question 1

Which of the following is NOT a key component of the Databricks Lakehouse architecture that differentiates it from traditional data lakes?

- A. ACID transactions support
- B. Schema enforcement capabilities
- C. Built-in machine learning model serving
- D. Time travel and versioning features
- E. Direct querying without ETL processes

Question 2

A data engineer needs to optimize cluster costs for a production ETL pipeline that runs every 2 hours and processes data for exactly 45 minutes each time. Which cluster configuration provides the most cost-effective solution?

- A. All-purpose cluster with auto-termination after 60 minutes
- B. Job cluster with spot instances enabled
- C. High-concurrency cluster with auto-scaling enabled
- D. Single-node cluster with on-demand instances
- E. All-purpose cluster with cluster pools

Question 3

When using Unity Catalog, which three-level namespace correctly represents the hierarchy for accessing a table named `customer_data` in schema `sales` within catalog `production`?

- A. `sales.production.customer_data` B. `production.sales.customer_data` C. `customer_data.sales.production` D. `production.customer_data.sales` E. `sales.customer_data.production`

Question 4

A Delta table has accumulated many small files over time, causing query performance to degrade. The table contains historical data that is frequently queried by date range. Which combination of commands will provide the best performance improvement?

- A. `OPTIMIZE table_name` followed by `VACUUM table_name` B. `OPTIMIZE table_name ZORDER BY (date_column)` followed by `VACUUM table_name RETAIN 168 HOURS` C. `VACUUM table_name` followed by `OPTIMIZE table_name` D. `ALTER TABLE table_name SET TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')` E. `REORG TABLE table_name`

Question 5

Which of the following SQL DDL statements correctly creates a Delta table with a check constraint that ensures the `price` column is always positive?

- A. `CREATE TABLE products (id INT, name STRING, price DECIMAL(10,2) CHECK (price > 0)) USING DELTA` B. `CREATE TABLE products (id INT, name STRING, price DECIMAL(10,2)) USING DELTA CONSTRAINT positive_price CHECK (price > 0)` C. `CREATE TABLE products (id INT, name STRING, price DECIMAL(10,2)) USING DELTA TBLPROPERTIES ('delta.constraints.positive_price' = 'price > 0')` D. `CREATE TABLE products (id INT, name STRING, price DECIMAL(10,2)) USING DELTA ADD CONSTRAINT positive_price CHECK (price > 0)` E. Check constraints are not supported in Delta tables

Question 6

A data engineer needs to implement a MERGE operation that updates existing records based on a composite key ((customer_id), (order_date)) and inserts new records. Which of the following MERGE statements is syntactically correct?

A.

sql

```
MERGE INTO orders target
USING new_orders source
ON target.customer_id = source.customer_id AND target.order_date = source.order_date
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

B.

sql

```
MERGE orders target
USING new_orders source
ON target.customer_id = source.customer_id AND target.order_date = source.order_date
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

C.

sql

```
MERGE INTO orders
FROM new_orders
ON orders.customer_id = new_orders.customer_id AND orders.order_date = new_orders.order_date
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

D.

```
sql

MERGE orders
WITH new_orders
ON orders.customer_id = new_orders.customer_id AND orders.order_date = new_orders.order_date
UPDATE WHEN MATCHED SET *
INSERT WHEN NOT MATCHED *
```

E.

```
sql

MERGE INTO orders target, new_orders source
WHERE target.customer_id = source.customer_id AND target.order_date = source.order_date
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

Question 7

In PySpark, which of the following methods correctly implements a custom aggregation function that calculates the coefficient of variation (standard deviation divided by mean) for a column?

A.

python

```
df.agg((stddev("value") / avg("value")).alias("cv"))
```

B.

python

```
df.select(stddev("value") / mean("value").alias("cv"))
```

C.

python

```
df.agg((stddev_samp("value") / mean("value")).alias("cv"))
```

D.

python

```
df.groupBy().agg((stddev("value") / avg("value")).alias("cv"))
```

E.

python

```
df.select(coefficient_of_variation("value").alias("cv"))
```

Question 8

A streaming DataFrame reads from a Kafka topic and needs to handle late-arriving events up to 2 hours after their event timestamp. Which watermark configuration is correct?

A.

```
python  
streaming_df.withWatermark("event_timestamp", "2 hours")
```

B.

```
python  
streaming_df.withWatermark("event_timestamp", "120 minutes")
```

C.

```
python  
streaming_df.watermark("event_timestamp", "2 hours")
```

D. Both A and B are correct

E.

```
python  
streaming_df.withLateTolerance("event_timestamp", "2 hours")
```

Question 9

When using Auto Loader to process JSON files from cloud storage, which configuration ensures that new columns are automatically added to the target schema without failing the stream?

A.

```
python
```

```
.option("cloudFiles.schemaEvolution", "addNewColumns")
```

B.

```
python
```

```
.option("cloudFiles.schemaEvolutionMode", "addNewColumns")
```

C.

```
python
```

```
.option("cloudFiles.inferSchema", "true")
```

D. Both A and B are correct

E.

```
python
```

```
.option("cloudFiles.allowSchemaEvolution", "true")
```

Question 10

A data engineer needs to process a DataFrame with nested JSON structures. Given the following schema:


```
root
|-- order_id: string
|-- items: array
    |-- element: struct
        |-- product_id: string
        |-- quantity: integer
        |-- price: double
```

Which PySpark transformation correctly flattens this structure to get individual item records?

A.

```
python

df.select("order_id", explode("items").alias("item")) \
    .select("order_id", "item.product_id", "item.quantity", "item.price")
```

B.

```
python

df.select("order_id", flatten("items").alias("item_list"))
```

C.

```
python

df.withColumn("item", explode("items")) \
    .select("order_id", "item.product_id", "item.quantity", "item.price")
```

D. Both A and C are correct

E.

```
python
```

```
df.select("order_id", unnest("items").alias("item"))
```

Question 11

In a Delta Live Tables pipeline, which expectation action should be used when data quality violations should be logged but records should still be included in the target table?

- A. `@dlt.expect("valid_data", "condition")` B. `@dlt.expect_or_fail("valid_data", "condition")` C. `@dlt.expect_or_drop("valid_data", "condition")` D. `@dlt.expect_or_quarantine("valid_data", "condition")` E. `@dlt.expect_all("valid_data", "condition")`

Question 12

Which of the following Delta Live Tables declarations correctly creates a streaming table that applies a change data capture pattern using the APPLY CHANGES INTO functionality?

A.

```
sql
```

```
CREATE STREAMING LIVE TABLE target_table
APPLY CHANGES FROM source_cdc_stream
KEYS (id)
SEQUENCE BY timestamp
```

B.

```
sql
```

```
CREATE OR REFRESH STREAMING LIVE TABLE target_table
APPLY CHANGES INTO STREAM(source_cdc_stream)
KEYS (id)
SEQUENCE BY timestamp
```

C.

```
sql

APPLY CHANGES INTO LIVE.target_table
FROM STREAM(LIVE.source_cdc_stream)
KEYS (id)
SEQUENCE BY timestamp
```

D.

```
sql

CREATE LIVE TABLE target_table
USING DELTA
SELECT * FROM STREAM(LIVE.source_cdc_stream)
```

E.

```
sql

CREATE STREAMING LIVE TABLE target_table (
  APPLY CHANGES FROM LIVE.source_cdc_stream
  KEYS (id)
  SEQUENCE BY timestamp
)
```

Question 13

A Structured Streaming job needs to write data to Delta Lake with exactly-once processing guarantees. Which combination of configurations is required?

- A. Setting a checkpoint location and using "append" output mode
- B. Setting a checkpoint location and using "complete" output mode
- C. Using "update" output mode with a watermark
- D. Setting idempotentWrites to true and using "append" mode
- E. Using foreachBatch with a custom write function

Question 14

When working with window functions in Spark SQL, which query correctly calculates a running total of sales partitioned by region and ordered by date?

A.

```
sql
SELECT region, date, sales,
SUM(sales) OVER (PARTITION BY region ORDER BY date ROWS UNBOUNDED PRECEDING) as running_total
FROM sales_data
```

B.

```
sql
SELECT region, date, sales,
SUM(sales) OVER (PARTITION BY region ORDER BY date RANGE UNBOUNDED PRECEDING) as running_total
FROM sales_data
```

C.

```
sql

SELECT region, date, sales,
SUM(sales) OVER (ORDER BY region, date ROWS UNBOUNDED PRECEDING) as running_total
FROM sales_data
```

D. Both A and B are correct

E.

```
sql

SELECT region, date, sales,
RUNNING_SUM(sales) OVER (PARTITION BY region ORDER BY date) as running_total
FROM sales_data
```

Question 15

Which of the following approaches correctly implements dynamic partition pruning optimization in a join query?

A. Using broadcast hints for both tables in the join B. Ensuring the partition column is included in the join condition C. Using bucketing on both tables with the same number of buckets D. Enabling adaptive query execution with `spark.sql.adaptive.enabled=true` E. Setting `spark.sql.optimizer.dynamicPartitionPruning.enabled=true` and having appropriate filter conditions

Question 16

A data engineer needs to read a CSV file with a custom delimiter (pipe "|") and handle quotes within fields. Which PySpark read operation is correctly configured?

A.

```
python  
spark.read.option("delimiter", "|").option("quote", "").csv(path)
```

B.

```
python  
spark.read.option("sep", "|").option("quote", "").option("escape", "").csv(path)
```

C.

```
python  
spark.read.format("csv").option("delimiter", "|").option("quoteChar", "").load(path)
```

D. Both B and C are correct

E.

```
python  
spark.read.csv(path, sep="|", quote="", escape="")
```

Question 17

When using Unity Catalog, which SQL command correctly grants SELECT permission on a specific table to a service principal?

A.

sql

```
GRANT SELECT ON TABLE catalog.schema.table_name TO SERVICE_PRINCIPAL 'sp-name'
```

B.

sql

```
GRANT SELECT ON catalog.schema.table_name TO `sp-name`
```

C.

sql

```
GRANT SELECT ON TABLE catalog.schema.table_name TO `sp-name`
```

D.

sql

```
GRANT SELECT ON catalog.schema.table_name TO SERVICE_PRINCIPAL `sp-name`
```

E.

sql

```
GRANT TABLE SELECT catalog.schema.table_name TO `sp-name`
```

Question 18

In a medallion architecture, a Bronze table contains raw IoT sensor data with potential duplicates and malformed records. Which of the following transformations is most appropriate for creating

the Silver layer?

- A. Simple data type casting and column renaming only
- B. Deduplication based on sensor_id and timestamp, data validation, and schema standardization
- C. Aggregating data by sensor and calculating summary statistics
- D. Joining with reference data and creating business-level metrics
- E. Archiving old records and keeping only the latest data points

Question 19

Which Databricks feature provides lineage tracking and impact analysis for data assets across the platform?

- A. Delta Lake transaction log
- B. Unity Catalog lineage capture
- C. MLflow experiment tracking
- D. Structured Streaming progress tracking
- E. Spark UI job dependencies

Question 20

A data pipeline processes files from cloud storage using Auto Loader. The pipeline needs to handle schema evolution while ensuring backwards compatibility. Which configuration approach is most appropriate?

- A. Using `cloudFiles.schemaEvolutionMode = "rescue"` to save malformed records
- B. Setting `cloudFiles.inferColumnTypes = "false"` to treat all new columns as strings
- C. Implementing a separate schema validation step before Auto Loader processing
- D. Using `cloudFiles.schemaEvolutionMode = "addNewColumns"` with rescue data column enabled
- E. Manually updating the schema definition file before processing new data

Question 21

When implementing a Type 2 Slowly Changing Dimension (SCD2) using Delta Lake, which MERGE statement pattern correctly handles both new records and updates to existing records?

A.

```
sql

MERGE INTO dim_table target
USING (
  SELECT *, current_timestamp() as effective_start_date,
         cast(null as timestamp) as effective_end_date,
         true as is_current
  FROM source_data
) source ON target.business_key = source.business_key AND target.is_current = true
WHEN MATCHED AND (target.column1 != source.column1 OR target.column2 != source.column2) THEN
  UPDATE SET effective_end_date = current_timestamp(), is_current = false
WHEN NOT MATCHED THEN INSERT *
```

B.

```
sql

MERGE INTO dim_table target
USING source_data source ON target.business_key = source.business_key
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

C. The MERGE operation alone is insufficient; requires a separate INSERT for new versions

D.

```
sql
```

```
MERGE INTO dim_table target
USING source_data source ON target.business_key = source.business_key AND target.is_current = true
WHEN MATCHED THEN UPDATE SET effective_end_date = current_timestamp(), is_current = false
WHEN NOT MATCHED THEN INSERT (business_key, column1, column2, effective_start_date, is_current)
VALUES (source.business_key, source.column1, source.column2, current_timestamp(), true)
```

E. SCD2 cannot be implemented efficiently with Delta Lake MERGE operations

Question 22

Which of the following correctly describes the relationship between RDDs, DataFrames, and Datasets in Spark's API evolution?

- A. DataFrames are built on top of RDDs, and Datasets are built on top of DataFrames
- B. RDDs, DataFrames, and Datasets are completely independent APIs
- C. Datasets are the lowest-level API, with DataFrames and RDDs built on top
- D. RDDs are deprecated and should not be used in modern Spark applications
- E. DataFrames and Datasets both provide structured APIs on top of RDDs, with Datasets adding compile-time type safety

Question 23

A streaming query processes events with the following requirements: calculate hourly aggregations, handle late data up to 30 minutes, and output results every 5 minutes. Which configuration correctly implements these requirements?

A.

```
python
```

```
streaming_df.withWatermark("event_time", "30 minutes") \
    .groupBy(window("event_time", "1 hour"), "category") \
    .agg(sum("value")) \
    .writeStream.trigger(processingTime="5 minutes")
```

B.

```
python

streaming_df.withWatermark("event_time", "30 minutes") \
    .groupBy(window("event_time", "1 hour", "5 minutes"), "category") \
    .agg(sum("value")) \
    .writeStream.trigger(processingTime="5 minutes")
```

C.

```
python

streaming_df.groupBy(window("event_time", "1 hour"), "category") \
    .agg(sum("value")) \
    .withWatermark("window", "30 minutes") \
    .writeStream.trigger(processingTime="5 minutes")
```

D.

```
python

streaming_df.withWatermark("event_time", "30 minutes") \
    .groupBy(tumbling_window("event_time", "1 hour"), "category") \
    .agg(sum("value")) \
    .writeStream.outputMode("update").trigger(processingTime="5 minutes")
```

E. Both A and D are correct

Question 24

In Delta Live Tables, which SQL syntax correctly creates a materialized view that refreshes automatically when upstream data changes?

A.

```
sql  
  
CREATE LIVE TABLE table_name AS  
SELECT * FROM LIVE.upstream_table WHERE condition
```

B.

```
sql  
  
CREATE MATERIALIZED VIEW table_name AS  
SELECT * FROM LIVE.upstream_table WHERE condition
```

C.

```
sql  
  
CREATE OR REFRESH LIVE TABLE table_name AS  
SELECT * FROM upstream_table WHERE condition
```

D.

```
sql
```

```
CREATE STREAMING LIVE TABLE table_name AS  
SELECT * FROM STREAM(LIVE.upstream_table) WHERE condition
```

E. Both A and C are correct

Question 25

Which of the following Spark configurations would most likely improve performance for a job that performs multiple joins on large datasets?

A.

```
spark.sql.adaptive.enabled=true  
spark.sql.adaptive.coalescePartitions.enabled=true  
spark.sql.adaptive.skewJoin.enabled=true
```

B.

```
spark.sql.autoBroadcastJoinThreshold=100MB  
spark.sql.shuffle.partitions=200
```

C.

```
spark.serializer=org.apache.spark.serializer.KryoSerializer  
spark.sql.execution.arrow.pyspark.enabled=true
```

D. All of the above configurations can help improve join performance

E.

```
spark.sql.crossJoin.enabled=true
spark.sql.optimizer.maxIterations=500
```

Question 26

A data engineer needs to implement a data quality check that validates email addresses using regex. Which Spark SQL function correctly implements this validation?

A.

```
sql

SELECT *, CASE WHEN email RLIKE '^ [A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'
  THEN true ELSE false END as valid_email
FROM user_data
```

B.

```
sql

SELECT *, regexp_like(email, '^ [A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$') as valid_email
FROM user_data
```

C.

```
sql

SELECT *, email ~ '^ [A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$' as valid_email
FROM user_data
```

D. Both A and B are correct

E.

```
sql
```

```
SELECT *, validate_email(email) as valid_email  
FROM user_data
```

Question 27

When using `foreachBatch` in Structured Streaming, which of the following statements is true about the batch processing function?

- A. The function receives a streaming `DataFrame` that processes data continuously
- B. The function receives a static `DataFrame` containing one micro-batch of data
- C. The function must return a `DataFrame` for the stream to continue
- D. The function automatically handles checkpointing and fault recovery
- E. The function can only perform write operations, not transformations

Question 28

Which Delta Lake feature allows you to query a table as it existed at a specific timestamp or version?

- A. Delta Log Replay
- B. Time Travel
- C. Version Control
- D. Snapshot Isolation
- E. Transaction Rollback

Question 29

A data engineer needs to optimize a query that frequently filters on multiple columns (`customer_id`, `order_date`, `status`). The table has 100 million rows. Which optimization technique

provides the best performance improvement?

- A. Creating a composite Z-ORDER index on all three columns
- B. Partitioning the table by customer_id and Z-ordering by order_date and status
- C. Creating separate tables for each status value
- D. Using Delta Lake's data skipping with appropriate statistics
- E. Both B and D together provide the best optimization

Question 30

In Unity Catalog, which of the following privilege hierarchies is correct?

- A. Metastore > Catalog > Schema > Table > Column
- B. Catalog > Schema > Table > Column (no metastore level)
- C. Workspace > Catalog > Schema > Table
- D. Metastore > Workspace > Catalog > Schema > Table
- E. Account > Metastore > Catalog > Schema > Table

Question 31

Which PySpark operation correctly pivots a DataFrame to convert rows to columns based on a category field?

A.

```
python  
  
df.pivot("category").agg(sum("value"))
```

B.

```
python
```



```
df.groupBy("id").pivot("category").agg(sum("value"))
```

C.

```
python
```

```
df.groupBy().pivot("category", ["A", "B", "C"]).sum("value")
```

D.

```
python
```

```
df.pivot("category", ["A", "B", "C"]).agg({"value": "sum"})
```

E. Both B and C are correct

Question 32

A streaming application needs to deduplicate records based on a composite key within each micro-batch while maintaining state across batches. Which approach is most appropriate?

- A. Using `dropDuplicates()` on the streaming DataFrame
- B. Using `dropDuplicates()` with watermark-based state cleanup
- C. Implementing custom deduplication logic in `foreachBatch`
- D. Using a Delta Lake MERGE operation in each micro-batch
- E. Both B and D are viable approaches

Question 33

Which of the following correctly describes the difference between `cache()` and `persist()` methods in Spark?

- A. cache() stores data in memory only, persist() allows choosing storage level
- B. cache() is synchronous, persist() is asynchronous
- C. persist() provides better performance than cache()
- D. cache() is deprecated in favor of persist()
- E. There is no functional difference between the two methods

Question 34

In Delta Live Tables, which approach correctly handles incremental processing of new files while maintaining exactly-once semantics?

- A. Using STREAMING LIVE TABLE with Auto Loader
- B. Using LIVE TABLE with file-based checkpointing
- C. Using scheduled batch processing with file tracking
- D. Using APPLY CHANGES INTO with merge semantics
- E. Both A and D can provide exactly-once processing

Question 35

A data pipeline needs to handle schema evolution in streaming JSON data where new fields may be added frequently. Which Auto Loader configuration provides the most robust solution?

A.

```
python

.option("cloudFiles.inferColumnTypes", "true")
.option("cloudFiles.schemaEvolutionMode", "addNewColumns")
```

B.

```
python
```

```
.option("cloudFiles.schemaEvolutionMode", "rescue")  
.option("rescuedDataColumn", "_rescued_data")
```

C.

```
python  
  
.option("cloudFiles.schemaEvolutionMode", "addNewColumns")  
.option("cloudFiles.schemaEvolutionMode", "rescue")  
.option("rescuedDataColumn", "_rescued_data")
```

D.

```
python  
  
.option("cloudFiles.inferSchema", "false")  
.option("mergeSchema", "true")
```

E. Schema evolution should be handled manually to ensure data quality

Question 36

Which SQL command correctly creates an external Delta table that references data stored in a specific cloud storage location?

A.

```
sql  
  
CREATE TABLE external_table  
USING DELTA  
LOCATION '/path/to/external/data'
```

B.

```
sql  
  
CREATE EXTERNAL TABLE external_table  
USING DELTA  
LOCATION '/path/to/external/data'
```

C.

```
sql  
  
CREATE TABLE external_table  
USING DELTA  
OPTIONS (path '/path/to/external/data')
```

D. Both A and C are correct

E.

```
sql  
  
CREATE OR REPLACE TABLE external_table  
USING DELTA  
EXTERNAL LOCATION '/path/to/external/data'
```

Question 37

When implementing error handling in a Structured Streaming application, which approach provides the most comprehensive error recovery mechanism?

A. Using try-catch blocks around the streaming query startup

B. Implementing custom error handling in foreachBatch with dead letter queue

- C. Setting streaming query restart policies at the cluster level
- D. Using watermarks to handle late and malformed data
- E. Enabling automatic query restart in Databricks Job configuration

Question 38

Which of the following Delta Lake operations requires the most careful consideration of concurrent access patterns?

- A. INSERT operations on partitioned tables
- B. UPDATE operations with complex predicate conditions
- C. VACUUM operations on frequently updated tables
- D. OPTIMIZE operations during high-traffic periods
- E. SELECT operations with time travel queries

Question 39

A data engineer needs to implement a real-time dashboard that shows aggregated metrics with 1-second latency. Which Databricks architectural approach is most suitable?

- A. Structured Streaming with continuous processing mode
- B. Delta Live Tables with triggered pipeline mode
- C. Databricks SQL with auto-refresh queries
- D. Real-time ML inference endpoints
- E. Structured Streaming with micro-batch processing every second

Question 40

In Unity Catalog, which of the following approaches correctly implements row-level security for a multi-tenant application?

- A. Creating separate tables for each tenant
- B. Using dynamic views with `current_user()` function filtering

- C. Implementing application-level filtering logic
- D. Using column-level permissions with tenant ID columns
- E. Creating separate schemas for each tenant with restricted access

Question 41

Which PySpark transformation correctly handles null values when performing aggregations to avoid skewed results?

A.

```
python  
  
df.filter(col("value").isNotNull()).groupBy("category").agg(avg("value"))
```

B.

```
python  
  
df.groupBy("category").agg(avg(when(col("value").isNotNull(), col("value"))))
```

C.

```
python  
  
df.na.drop(subset=["value"]).groupBy("category").agg(avg("value"))
```

D. All of the above approaches handle nulls correctly

E.

```
python
```

```
df.groupBy("category").agg(avg(coalesce(col("value"), lit(0))))
```

Question 42

When designing a data pipeline for CDC (Change Data Capture) processing, which pattern provides the most efficient and scalable solution?

- A. Full table replication on every change
- B. Delta Lake MERGE operations with source change logs
- C. Separate tables for inserts, updates, and deletes
- D. Event sourcing with complete transaction history
- E. Hybrid approach combining batch and streaming processing

Question 43

Which Databricks Jobs configuration provides the best balance of cost optimization and reliability for a critical daily ETL pipeline?

- A. Single task job with maximum retry count
- B. Multiple task job with linear dependencies and per-task retry policies
- C. Multiple task job with parallel execution and job-level retry
- D. Single task job with cluster auto-scaling and spot instances
- E. Multiple task job with conditional task execution based on upstream success

Question 44

A data engineer needs to join a large fact table (1TB) with a small dimension table (10MB) that fits in memory. Which join strategy optimization provides the best performance?

- A.

```
python
```

```
large_df.join(broadcast(small_df), "join_key")
```

B.

```
python  
large_df.join(small_df.hint("broadcast"), "join_key")
```

C.

```
python  
large_df.join(small_df, "join_key").hint("broadcast", "small_df")
```

D. Both A and B are correct

E.

```
python  
large_df.broadcastJoin(small_df, "join_key")
```

Question 45

In Delta Live Tables, which expectation configuration correctly implements a data quality rule that quarantines invalid records while allowing the pipeline to continue processing valid records?

A.

```
python  
@dlt.expect("valid_email", "email RLIKE '^[^@]+@[^@]+\.[^@]+$'")
```


B.

python

```
@dlt.expect_or_drop("valid_email", "email RLIKE '^[^@]+@[^@]+\.[^@]+$'")
```

C.

python

```
@dlt.expect_or_fail("valid_email", "email RLIKE '^[^@]+@[^@]+\.[^@]+$'")
```

D.

python

```
@dlt.expect_all_or_drop("valid_email", "email RLIKE '^[^@]+@[^@]+\.[^@]+$'")
```

E. Quarantining requires a separate quarantine table definition

Question 46

Which of the following Spark SQL window functions correctly calculates the percentage contribution of each row to the total sum within its partition?

A.

sql

```
SELECT *, value / SUM(value) OVER (PARTITION BY category) * 100 as percentage  
FROM sales_data
```

B.

sql

```
SELECT *, PERCENT_RANK() OVER (PARTITION BY category ORDER BY value) * 100 as percentage  
FROM sales_data
```

C.

sql

```
SELECT *, value / SUM(value) OVER () * 100 as percentage  
FROM sales_data
```

D.

sql

```
SELECT *, RATIO_TO_REPORT(value) OVER (PARTITION BY category) * 100 as percentage  
FROM sales_data
```

E. Both A and D are correct

Question 47

A streaming pipeline processes IoT sensor data and needs to detect anomalies based on statistical patterns. Which approach provides the most scalable real-time anomaly detection?

- A. Pre-computing statistical thresholds and using streaming filters
- B. Implementing machine learning models in foreachBatch processing
- C. Using complex event processing with time-based windows
- D. Storing all data first and running batch anomaly detection
- E. Using Structured Streaming with stateful operations for rolling statistics

Question 48

When implementing a data lakehouse architecture, which storage layer pattern provides the best balance of performance, cost, and governance?

- A. Raw data in object storage, processed data in Delta Lake
- B. All data in Delta Lake with different optimization strategies per layer
- C. Hot data in Delta Lake, cold data in object storage with lifecycle policies
- D. Structured data in Delta Lake, unstructured data in object storage
- E. Real-time data in memory, batch data in Delta Lake, archival in object storage

Question 49

Which Unity Catalog feature provides automated discovery and cataloging of data assets across the organization?

- A. Data lineage tracking
- B. Automatic schema inference
- C. Discovery and tagging workflows
- D. Information schema views
- E. System table analytics

Question 50

A data pipeline needs to process files with varying schemas where column order and names may change. Which approach provides the most robust solution?

- A. Using schema hints with column position mapping
- B. Implementing schema evolution with rescue data columns
- C. Using column name-based mapping with case-insensitive matching
- D. Pre-processing files to standardize schema before ingestion
- E. Using Auto Loader with flexible schema inference and evolution

Question 51

Which Delta Lake transaction isolation level ensures that concurrent operations do not interfere with each other?

- A. Read Uncommitted
- B. Read Committed
- C. Repeatable Read
- D. Serializable
- E. Snapshot Isolation

Question 52

A data engineer needs to implement a custom UDF (User Defined Function) that processes arrays of structs. Which approach provides the best performance in PySpark?

- A. Python UDF with native Python processing
- B. Pandas UDF with vectorized operations
- C. Scala UDF compiled to JVM bytecode
- D. SQL expression using built-in array functions
- E. Both C and D provide optimal performance

Question 53

When using Databricks Auto Loader for incremental file processing, which trigger configuration processes files immediately as they arrive?

- A. `.trigger(once=True)`
- B. `.trigger(processingTime="0 seconds")`
- C. `.trigger(continuous="1 second")`
- D. `.trigger(availableNow=True)`
- E. Auto Loader automatically triggers on file arrival

Question 54

Which of the following correctly implements a streaming aggregation that maintains state for session-based analysis?

A.

```
python  
  
streaming_df.withWatermark("timestamp", "30 minutes") \  
    .groupBy("user_id", session_window("timestamp", "30 minutes")) \  
    .agg(count("*").alias("events_per_session"))
```

B.

```
python  
  
streaming_df.withWatermark("timestamp", "30 minutes") \  
    .groupBy("user_id", window("timestamp", "30 minutes")) \  
    .agg(count("*").alias("events_per_session"))
```

C.

```
python  
  
streaming_df.groupBy("user_id") \  
    .agg(count("*").alias("total_events")) \  
    .withWatermark("timestamp", "30 minutes")
```

D.

```
python
```

```
streaming_df.withWatermark("timestamp", "30 minutes") \
    .groupBy("user_id") \
    .agg(approxCountDistinct("session_id").alias("unique_sessions"))
```

E. Session-based windowing is not supported in Structured Streaming

Question 55

In Unity Catalog, which SQL command correctly creates a service principal and grants it appropriate permissions for automated data processing?

A.

```
sql

CREATE SERVICE PRINCIPAL 'etl-service-principal';
GRANT SELECT, INSERT ON SCHEMA production.sales TO 'etl-service-principal';
```

B. Service principals must be created through the Databricks UI, not SQL

C.

```
sql

CREATE USER SERVICE PRINCIPAL 'etl-service-principal';
GRANT USE CATALOG, USE SCHEMA, SELECT, INSERT ON production.sales TO 'etl-service-principal';
```

D.

```
sql
```

```
ALTER CATALOG production ADD SERVICE PRINCIPAL 'etl-service-principal';  
GRANT USAGE ON CATALOG production TO 'etl-service-principal';
```

E. Both A and C are valid approaches

Question 56

Which optimization technique provides the most significant performance improvement for queries that frequently filter on high-cardinality string columns?

- A. Bloom filters on the filter columns
- B. Z-ordering the table by the filter columns
- C. Partitioning the table by hash of the filter columns
- D. Creating dictionary encoding for string values
- E. Using column store format with compression

Question 57

A Delta Live Tables pipeline needs to handle both streaming and batch data sources in the same workflow. Which pattern correctly implements this mixed-mode processing?

A.

```
sql
```

```
CREATE STREAMING LIVE TABLE streaming_bronze AS  
SELECT * FROM cloud_files('/path/to/stream')
```

```
CREATE LIVE TABLE batch_bronze AS  
SELECT * FROM '/path/to/batch'
```

```
CREATE LIVE TABLE silver AS  
SELECT * FROM LIVE.streaming_bronze  
UNION ALL  
SELECT * FROM LIVE.batch_bronze
```

B.

```
sql
```

```
CREATE OR REFRESH STREAMING LIVE TABLE mixed_bronze AS  
SELECT * FROM cloud_files('/path/to/stream')  
UNION ALL  
SELECT * FROM '/path/to/batch'
```

C. Mixed streaming and batch sources require separate pipelines

D.

```
sql
```

```
CREATE LIVE TABLE silver AS  
SELECT * FROM STREAM(LIVE.streaming_bronze)  
UNION ALL  
SELECT * FROM LIVE.batch_bronze
```

E. Use APPLY CHANGES INTO to merge streaming and batch data

Question 58

Which PySpark configuration correctly enables adaptive query execution for dynamic partition pruning and join optimization?

A.

```
python  
  
spark.conf.set("spark.sql.adaptive.enabled", "true")  
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
```

B.

```
python  
  
spark.conf.set("spark.sql.adaptive.enabled", "true")  
spark.conf.set("spark.sql.optimizer.dynamicPartitionPruning.enabled", "true")  
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
```

C.

```
python  
  
spark.conf.set("spark.sql.adaptive.queryExecution.enabled", "true")  
spark.conf.set("spark.sql.adaptive.join.enabled", "true")
```

D. Both A and B together provide comprehensive adaptive optimization

E. Adaptive query execution is enabled by default in Databricks

Question 59

When implementing a real-time ML feature store, which architecture pattern provides the best performance for both training and serving workloads?

- A. Delta Lake tables with feature serving APIs
- B. In-memory feature store with persistent backup
- C. Streaming feature computation with cached results
- D. Pre-computed feature tables with real-time updates
- E. Hybrid batch and streaming feature computation with unified serving layer

Question 60

Which approach correctly implements incremental data loading with change detection for a slowly changing dimension table?

A.

```
sql

MERGE INTO dim_customer target
USING (
  SELECT *, md5(concat_ws('||', col1, col2, col3)) as hash_key
  FROM source_customer
) source ON target.customer_id = source.customer_id
WHEN MATCHED AND target.hash_key != source.hash_key THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

B.

```
sql

INSERT OVERWRITE dim_customer
SELECT * FROM source_customer
WHERE last_modified > (SELECT max(last_modified) FROM dim_customer)
```

C.

sql

```
DELETE FROM dim_customer WHERE customer_id IN (SELECT customer_id FROM source_customer);  
INSERT INTO dim_customer SELECT * FROM source_customer;
```

D. Option A is the most efficient approach for change detection

E.

sql

```
CREATE OR REPLACE TABLE dim_customer AS  
SELECT * FROM source_customer
```

Question 61

In Structured Streaming, which output mode is required when using non-time-based stateful operations like mapGroupsWithState?

- A. append
- B. update
- C. complete
- D. Any output mode can be used
- E. Custom output mode must be defined

Question 62

Which Delta Live Tables constraint syntax correctly implements a referential integrity check between tables?

A.

sql

```
CREATE LIVE TABLE orders (  
  order_id INT,  
  customer_id INT,  
  CONSTRAINT valid_customer EXPECT (customer_id IN (SELECT customer_id FROM LIVE.customers))  
)
```

B.

sql

```
CREATE LIVE TABLE orders (  
  order_id INT,  
  customer_id INT,  
  CONSTRAINT fk_customer FOREIGN KEY (customer_id) REFERENCES LIVE.customers(customer_id)  
)
```

C.

sql

```
CREATE LIVE TABLE orders (  
  order_id INT,  
  customer_id INT,  
  CONSTRAINT valid_customer EXPECT EXISTS (SELECT 1 FROM LIVE.customers WHERE customer_id = orders.c  
)
```



D. Referential integrity must be enforced at the application level

E.

sql

```
CREATE LIVE TABLE orders (  
  order_id INT,  
  customer_id INT  
) WITH REFERENTIAL_INTEGRITY (customer_id -> LIVE.customers.customer_id)
```

Question 63

A data pipeline processes nested JSON with deeply nested arrays and maps. Which approach provides the best performance for flattening and querying this data?

- A. Using explode() functions recursively for each level
- B. Converting to Delta Lake with nested column pruning optimization
- C. Flattening all data at ingestion time into a wide table structure
- D. Using schema inference with selective column reading
- E. Implementing custom flatten UDFs with optimized serialization

Question 64

Which Databricks SQL Warehouse configuration provides the most cost-effective solution for ad-hoc analytical queries with unpredictable usage patterns?

- A. Always-on cluster with autoscaling enabled
- B. Serverless SQL Warehouse with automatic pause/resume
- C. Classic SQL Endpoint with spot instances
- D. Multi-cluster warehouse with query queueing
- E. Photon-accelerated warehouse with reserved capacity

Question 65

When implementing data lineage tracking in Unity Catalog, which approach captures the most comprehensive lineage information?

- A. Manual lineage registration through APIs
- B. Automatic lineage capture from Spark execution plans
- C. Custom lineage tracking in application code
- D. Lineage inference from table dependencies
- E. Integration with external data catalog tools

Question 66

Which PySpark operation correctly implements a complex aggregation that calculates both rolling averages and cumulative sums within the same query?

A.

```
python

from pyspark.sql.window import Window
window_spec = Window.partitionBy("category").orderBy("date")
df.withColumn("rolling_avg", avg("value").over(window_spec.rowsBetween(-6, 0))) \
  .withColumn("cumulative_sum", sum("value").over(window_spec.rowsBetween(Window.unboundedPreceding, 0)))
```

B.

```
python

df.groupBy("category", "date") \
  .agg(avg("value").over(Window.orderBy("date").rowsBetween(-6, 0)).alias("rolling_avg"),
       sum("value").over(Window.orderBy("date")).alias("cumulative_sum"))
```

C.

```
python
```

```
df.withColumn("rolling_avg", avg("value").over(Window.partitionBy("category").orderBy("date").rangeBetween  
    .withColumn("cumulative_sum", sum("value").over(Window.partitionBy("category").orderBy("date"))))
```

D. Both A and C are correct implementations

E. Rolling aggregations require separate queries for optimal performance

Question 67

In a medallion architecture implementation, which data quality pattern provides the most comprehensive validation at the Bronze to Silver transition?

A. Schema validation only

B. Null value checking and data type validation

C. Completeness, validity, uniqueness, and consistency checks

D. Statistical outlier detection

E. Business rule validation against reference data

Question 68

Which Auto Loader configuration provides the most efficient processing for large files (>1GB) with nested JSON structures?

A.

```
python  
  
.option("cloudFiles.format", "json")  
.option("multiLine", "true")  
.option("cloudFiles.maxFilesPerTrigger", "1")
```

B.

python

```
.option("cloudFiles.format", "json")  
.option("cloudFiles.maxBytesPerTrigger", "1GB")  
.option("cloudFiles.includeExistingFiles", "false")
```

C.

python

```
.option("cloudFiles.format", "json")  
.option("recursiveFileLookup", "true")  
.option("pathGlobFilter", "*.json")
```

D.

python

```
.option("cloudFiles.format", "text")  
.option("wholetext", "true")  
.option("cloudFiles.maxFilesPerTrigger", "1")
```

E. Large JSON files should be pre-processed before Auto Loader ingestion

Question 69

Which Delta Lake feature provides the most efficient way to handle DELETE operations on large tables with complex filter conditions?

- A. Standard DELETE with optimized predicates
- B. MERGE operation with WHEN MATCHED THEN DELETE
- C. CREATE TABLE AS SELECT excluding deleted records

- D. Partition-level deletion with metadata-only operations
- E. Using Z-ordering on delete predicate columns before DELETE

Question 70

A streaming job needs to join streaming data with a slowly changing dimension table. Which pattern provides the most efficient and consistent results?

- A. Stream-to-stream join with watermarks
- B. Stream-to-static join with Delta Lake table
- C. Broadcast join with cached dimension data
- D. Temporal join with versioned dimension history
- E. Both B and D provide consistency guarantees

Question 71

Which Unity Catalog governance feature provides automated data classification and sensitive data discovery?

- A. Column-level lineage tracking
- B. Automatic data profiling and tagging
- C. Schema evolution monitoring
- D. Query access pattern analysis
- E. Data quality metric collection

Question 72

In Delta Live Tables, which approach correctly implements conditional pipeline execution based on upstream data availability?

- A.

```
python
```

```
@dlt.table(
    condition="SELECT COUNT(*) > 0 FROM LIVE.source_table"
)
def target_table():
    return spark.sql("SELECT * FROM LIVE.source_table")
```

B. Conditional execution requires custom pipeline orchestration

C.

```
python

@dlt.table
def target_table():
    if spark.sql("SELECT COUNT(*) FROM LIVE.source_table").collect()[0][0] > 0:
        return spark.sql("SELECT * FROM LIVE.source_table")
    else:
        return spark.sql("SELECT * FROM LIVE.empty_schema_table WHERE FALSE")
```

D.

```
sql

CREATE LIVE TABLE target_table AS
SELECT * FROM LIVE.source_table
WHERE EXISTS (SELECT 1 FROM LIVE.source_table)
```

E. Use expectations to validate data availability before processing

Question 73

Which Spark SQL optimization provides the most significant performance improvement for star schema queries with multiple dimension table joins?

- A. Broadcast join hints for all dimension tables
- B. Bucketing fact and dimension tables on join keys
- C. Star schema optimization with dimension table broadcast
- D. Denormalizing dimensions into the fact table
- E. Using columnar storage format for all tables

Question 74

A data engineer needs to implement exactly-once semantics for a streaming ETL pipeline that writes to multiple downstream systems. Which architecture pattern provides the strongest guarantees?

- A. Idempotent writes with unique transaction IDs
- B. Two-phase commit protocol across all systems
- C. Delta Lake ACID transactions with external system coordination
- D. Checkpoint-based recovery with duplicate detection
- E. Event sourcing with compensating transactions

Question 75

Which PySpark DataFrame operation correctly implements a complex transformation that requires accessing both current and previous row values?

A.

```
python
from pyspark.sql.window import Window
window_spec = Window.partitionBy("id").orderBy("timestamp")
df.withColumn("prev_value", lag("value", 1).over(window_spec)) \
  .withColumn("value_change", col("value") - col("prev_value"))
```

B.

python

```
df.withColumn("prev_value", lag("value").over(Window.orderBy("timestamp"))) \
.withColumn("value_change", col("value") - col("prev_value"))
```

C.

python

```
df.selectExpr("*, 'value - lag(value, 1) OVER (PARTITION BY id ORDER BY timestamp) as value_change'")
```

D. Both A and C are correct

E. Previous row access requires iterative DataFrame processing

Question 76

When implementing a data retention policy in Delta Lake, which approach provides the most efficient cleanup while maintaining query performance?

- A. Using VACUUM with appropriate retention periods
- B. Partitioning by date and dropping old partitions
- C. Archiving old data to cheaper storage tiers
- D. Using DELETE operations with time-based predicates
- E. Combining partitioning, VACUUM, and archival strategies

Question 77

Which Delta Live Tables monitoring capability provides real-time visibility into pipeline execution and data quality metrics?

- A. System table queries for pipeline events
- B. Built-in monitoring dashboards and alerts
- C. Custom metrics collection through expectations
- D. Integration with external monitoring systems
- E. All of the above provide comprehensive monitoring

Question 78

A streaming application processes high-velocity event data and needs to detect complex event patterns across multiple event types. Which approach provides the most scalable solution?

- A. Complex event processing with state management
- B. Time-based windowing with pattern matching
- C. Machine learning-based pattern detection
- D. Rule-based stream processing with joins
- E. Hybrid approach combining windowing and state management

Question 79

Which Unity Catalog feature enables fine-grained access control at the row and column level for sensitive data?

- A. Dynamic data masking functions
- B. Row-level security policies with SQL predicates
- C. Column-level encryption with user-based keys
- D. Attribute-based access control (ABAC)
- E. Both A and B provide fine-grained control

Question 80

When optimizing Spark jobs for large-scale data processing, which memory management configuration provides the best balance of performance and stability?

A.

```
spark.executor.memory=8g  
spark.executor.memoryFraction=0.8  
spark.storage.memoryFraction=0.5
```

B.

```
spark.sql.adaptive.enabled=true  
spark.sql.adaptive.coalescePartitions.enabled=true  
spark.sql.adaptive.advisoryPartitionSizeInBytes=128MB
```

C.

```
spark.serializer=org.apache.spark.serializer.KryoSerializer  
spark.executor.memory=16g  
spark.executor.cores=4
```

D. Memory configuration depends on workload characteristics and cluster resources

E.

```
spark.dynamicAllocation.enabled=true  
spark.dynamicAllocation.maxExecutors=100  
spark.sql.shuffle.partitions=400
```

Question 81

Which approach correctly implements a custom data source connector for reading from a proprietary file format in Databricks?

- A. Implementing DataSource V2 API with custom reader
- B. Creating a Python UDF wrapper around existing libraries
- C. Using Spark's binary file reader with custom parsing logic
- D. Developing a native Scala connector with DataFrame integration
- E. Both A and D provide full integration capabilities

Question 82

In a multi-tenant data platform, which partitioning strategy provides the best isolation and performance for tenant-specific queries?

- A. Hash partitioning by tenant ID
- B. Range partitioning by tenant ID and date
- C. Directory-based partitioning with tenant isolation
- D. Separate tables per tenant with shared schema
- E. Hybrid approach with logical and physical partitioning

Question 83

Which Delta Lake feature provides automatic schema evolution and conflict resolution for concurrent writers?

- A. Optimistic concurrency control with schema merging
- B. Schema evolution with backward compatibility checks
- C. Automatic schema inference with conflict detection
- D. Writer coordination through transaction log
- E. Schema evolution must be managed explicitly by applications

Question 84

A data pipeline needs to process both real-time streaming data and large batch corrections. Which architecture pattern handles both workloads efficiently?

- A. Lambda architecture with separate batch and streaming layers
- B. Kappa architecture with unified stream processing
- C. Delta Lake with streaming writes and batch MERGE operations
- D. Microservice architecture with specialized processing services
- E. Event-driven architecture with message queues

Question 85

Which PySpark optimization technique provides the most significant performance improvement for iterative machine learning algorithms?

- A. DataFrame caching with appropriate storage levels
- B. RDD persistence with memory and disk spill
- C. Broadcast variables for large lookup tables
- D. Partitioning optimization for data locality
- E. All optimizations should be applied together for iterative workloads

Question 86

When implementing data lineage in Unity Catalog, which metadata capture approach provides the most complete visibility?

- A. Automatic lineage from Spark SQL execution plans
- B. Custom lineage registration through REST APIs
- C. Integration with external workflow orchestration tools
- D. Manual documentation of data transformations
- E. Combining automatic capture with manual annotations for business context

Question 87

Which Databricks feature provides the most comprehensive solution for managing ML model lifecycle from development to production?

- A. MLflow with model registry and serving
- B. Feature Store with automated feature engineering
- C. AutoML with automated model selection
- D. Model serving endpoints with A/B testing
- E. Integrated MLOps platform combining all components

Question 88

A streaming job processes IoT sensor data and needs to handle out-of-order events with complex temporal patterns. Which windowing strategy provides the most accurate results?

- A. Tumbling windows with watermarks
- B. Sliding windows with late data tolerance
- C. Session windows with dynamic timeout
- D. Custom windowing with stateful processing
- E. Combination of multiple windowing strategies

Question 89

Which Unity Catalog audit capability provides the most comprehensive tracking of data access and usage patterns?

- A. Query history with user attribution
- B. System table analytics for access patterns
- C. Real-time access monitoring with alerts
- D. Compliance reporting with automated generation
- E. Comprehensive audit logging across all access vectors

Question 90

When implementing a data quality framework, which approach provides the most scalable validation across large datasets?

- A. Rule-based validation with parallel processing
- B. Statistical profiling with anomaly detection
- C. Machine learning-based quality scoring
- D. Sampling-based validation with confidence intervals
- E. Multi-layered validation combining multiple approaches

Question 91

Which Delta Lake transaction feature provides the strongest consistency guarantees for concurrent read-write workloads?

- A. Snapshot isolation with MVCC
- B. Serializable isolation with conflict detection
- C. Read committed isolation with row-level locking
- D. Optimistic concurrency control with retry logic
- E. Delta Lake automatically provides ACID guarantees

Question 92

A data pipeline needs to integrate with legacy systems that provide data through various protocols (FTP, SFTP, HTTP, database connections). Which integration pattern provides the most maintainable solution?

- A. Custom connectors for each protocol
- B. Universal adapter pattern with protocol abstraction
- C. ETL tool integration with protocol support
- D. API gateway for protocol translation
- E. Microservice architecture with specialized connectors

Question 93

Which Spark SQL feature provides the most efficient execution for queries with complex nested subqueries and correlated predicates?

- A. Subquery optimization with predicate pushdown
- B. Common table expressions (CTEs) for query restructuring
- C. Adaptive query execution with runtime optimization
- D. Manual query rewriting with joins
- E. Cost-based optimizer with statistics collection

Question 94

When designing a disaster recovery strategy for a Databricks lakehouse, which approach provides the most comprehensive protection?

- A. Cross-region Delta Lake table replication
- B. Automated backup and restore procedures
- C. Multi-cloud deployment with active-passive failover
- D. Point-in-time recovery with transaction log preservation
- E. Comprehensive strategy combining multiple approaches

Question 95

Which Auto Loader feature provides the most efficient handling of schema evolution in high-volume streaming scenarios?

- A. Automatic schema inference with evolution tracking
- B. Schema hints with manual approval workflows
- C. Rescue data columns with schema validation
- D. Streaming schema evolution with conflict resolution
- E. Schema registry integration with version control

Question 96

A data engineer needs to implement a complex business rule that requires accessing data from multiple tables with temporal constraints. Which SQL approach provides the most efficient execution?

- A. Complex joins with window functions
- B. Correlated subqueries with EXISTS clauses
- C. Common table expressions with recursive logic
- D. Materialized views with incremental refresh
- E. Stored procedures with optimized query plans

Question 97

Which Databricks workspace governance feature provides the most comprehensive control over resource usage and cost management?

- A. Cluster policies with resource limits
- B. Unity Catalog with usage-based billing allocation
- C. Workspace-level access controls with budget alerts
- D. Usage analytics with automated reporting
- E. Comprehensive governance combining all features

Question 98

When implementing real-time fraud detection, which stream processing pattern provides the lowest latency while maintaining high accuracy?

- A. Complex event processing with pattern matching
- B. Machine learning inference on streaming data
- C. Rule-based filtering with statistical thresholds
- D. Hybrid batch-streaming with model updates
- E. Edge processing with cloud-based model serving

Question 99

Which Delta Live Tables configuration provides the most efficient processing for pipelines with multiple branching paths and conditional logic?

- A. Linear pipeline with conditional expectations
- B. Graph-based pipeline with dynamic dependencies
- C. Parallel pipeline execution with result merging
- D. Hierarchical pipeline with staged processing
- E. Event-driven pipeline with trigger-based execution

Question 100

A data platform needs to support both OLTP-style operations and OLAP-style analytics on the same dataset. Which Databricks architecture provides the most efficient solution for this hybrid workload?

- A. Delta Lake with optimized read/write patterns
- B. Separate OLTP and OLAP systems with data synchronization
- C. In-memory processing with persistent storage
- D. Columnar storage with row-level operations support
- E. Lakehouse architecture with workload-specific optimizations

Correct Answers

- 1. C
- 2. B
- 3. B
- 4. B
- 5. C

- 6. A
- 7. D
- 8. D
- 9. D
- 10. D
- 11. A
- 12. C
- 13. A
- 14. A
- 15. E
- 16. D
- 17. C
- 18. B
- 19. B
- 20. E
- 21. C
- 22. E
- 23. A
- 24. E
- 25. D
- 26. D
- 27. B
- 28. B

29. E

30. E

31. B

32. E

33. A

34. E

35. C

36. D

37. B

38. C

39. A

40. B

41. D

42. B

43. B

44. D

45. B

46. E

47. E

48. E

49. C

50. E

51. E

52. E

53. D

54. A

55. B

56. B

57. A

58. D

59. E

60. D

61. B

62. A

63. C

64. B

65. B

66. A

67. C

68. A

69. B

70. E

71. B

72. E

73. C

74. C

75. D

76. E

77. E

78. E

79. E

80. D

81. E

82. E

83. B

84. C

85. E

86. E

87. E

88. E

89. E

90. E

91. A

92. B

93. C

94. E

95. E

96. A

97. E

98. E

99. B

100. E

Study Guide and Tips

Recommended Study Approach:

1. Practice Hands-On Labs

- Set up a Databricks Community Edition account
- Practice writing Delta Lake operations, streaming queries, and DLT pipelines
- Experiment with Unity Catalog features and permissions

2. Focus Areas by Domain:

- **Lakehouse Platform (15%):** Unity Catalog, cluster management, workspace features
- **ELT with Spark SQL/Python (30%):** DataFrames, SQL operations, transformations, joins
- **Incremental Processing (20%):** Auto Loader, streaming, watermarks, CDC patterns
- **Production Pipelines (20%):** Delta Live Tables, Jobs, monitoring, error handling
- **Data Governance (15%):** Security, permissions, lineage, compliance

3. Key Topics to Master:

- Delta Lake ACID operations (MERGE, OPTIMIZE, VACUUM)
- Structured Streaming concepts (watermarks, triggers, output modes)
- Delta Live Tables syntax and patterns
- Unity Catalog three-level namespace and permissions
- Performance optimization techniques

4. Time Management:

- Practice completing questions in 72 seconds average (120 minutes ÷ 100 questions)
- Mark difficult questions for review
- Focus on eliminating obviously wrong answers first

5. Common Pitfalls to Avoid:

- Confusing streaming vs. batch processing syntax
- Mixing up Delta Live Tables decorators and SQL syntax
- Forgetting proper three-level namespace format in Unity Catalog
- Overlooking checkpoint locations in streaming queries

Final Exam Preparation:

- Take this practice exam under timed conditions
- Review incorrect answers thoroughly
- Focus additional study on topics where you scored lowest
- Practice explaining concepts out loud to solidify understanding

Good luck with your Databricks Certified Data Engineer Associate exam!